

程序设计语言初步

-控制结构

软件与理论中心 张艳梅



◆ 计算机中的指令除了运算，还需要控制语句的执行序列，如：是非选择、多项选择、周而复始的循环、中断、意外终止等。

◆ 在写程序时，有时往往需要测试某一个条件是否成立，然后根据测试结果来控制程序后续执行路径。此时要用到选择结构。

◆ C语言中选择语句有：

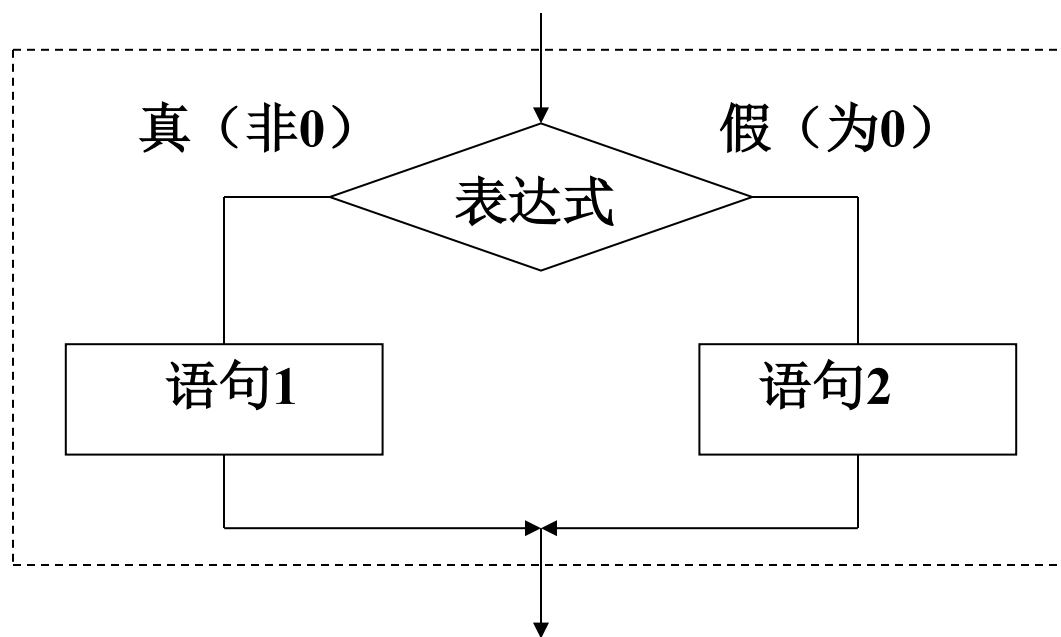
1) **if-else** 语句 (是非选择，两种分支)

2) **switch-case**语句 (多路选择)

是非选择结构

if-else

实现双路选择



a) 流程图

```
if (表达式)
    语句1
else
    语句2
```

b) C语法

用某个表达式单独测试每一个可能的整数值常量，然后做出相应的处理。

1. switch语句的一般形式

```
switch (表达式) { // 计算表达式的值
    case 常量表达式1: 语句组; break;
    // 等于常量式1时的处理语句组，无需加{}
    case 常量表达式2: 语句组; break;
    .....
    case 常量表达式 n: 语句组; break;
    [default: 语句组; [break; ]] // 表达式≠常量
    1..n时处理
}
```

如何使完成的程序重复执行呢？

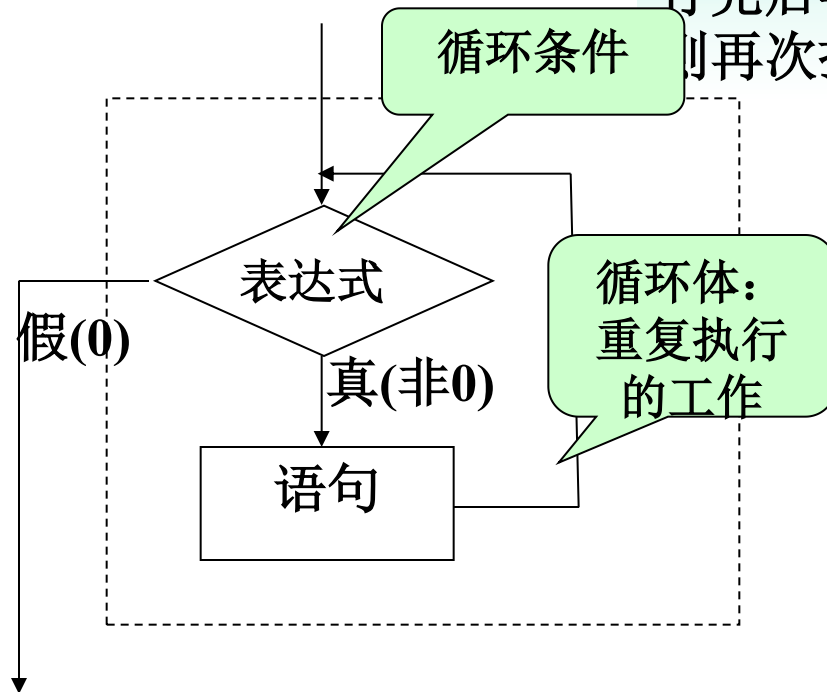
四、循环结构语句

- ◆ 有时程序中需要多次运行同一段代码（重复做相同的事情）。这种控制结构称为循环结构。
- ◆ 在循环结构中，需要刻画出重复执行的是哪些动作（循环体），以及什么条件下需要重复（循环条件）。
- ◆ C语言提供了三类用于实现循环结构的语句：
 - 1) **while**
 - 2) **do-while**
 - 3) **for**

此处的表达式可以为任何一种表达式,语句为任意类语句;

如果表达式值为真(非0), 则执行语句; 执行完后再测试表达式, 如果仍为真(非0), 则再次执行语句; 直到表达式为假(为0)。

while



a) 流程图

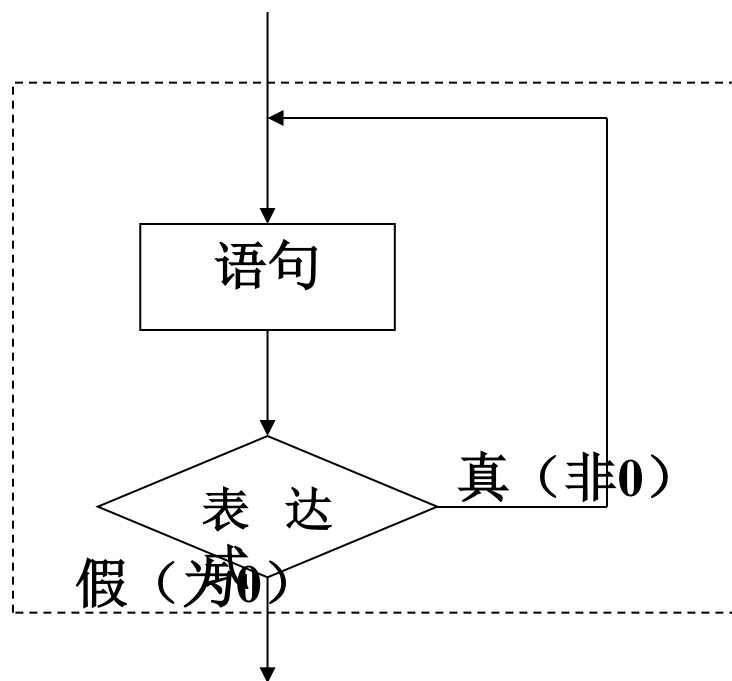
```
while (表达式)
    语句
```

b) C语句

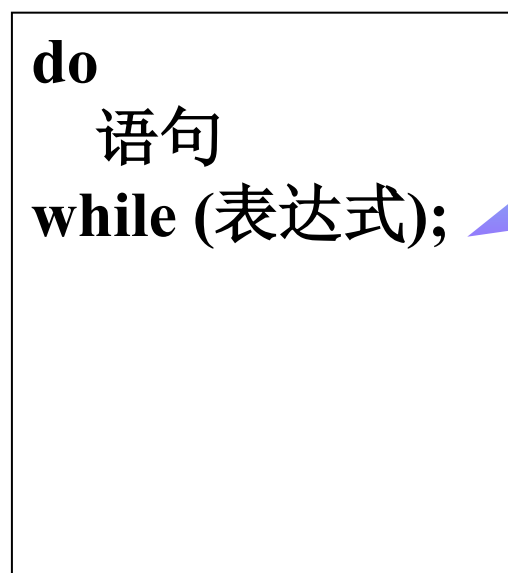
图3. 6 while语句

do while循环结构

do-while循环结构，先执行一次再循环判断。



a) 流程图



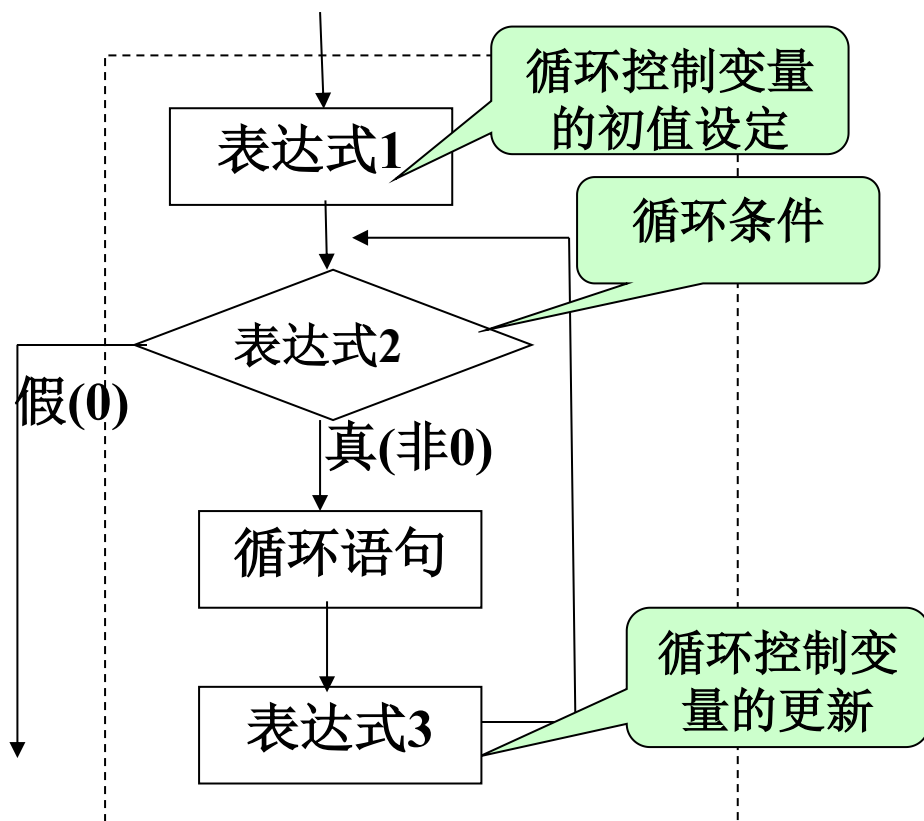
注意此处
有语句结
束符;

b) C语句

图3. 7 do-while语句

最简洁的循环结构: for循环

for(表达式1;表达式2;表达式3) 循环语句;



与仅强调循环条件的 **while(表达式2)** 循环结构相比，凸显了循环控制的全部要素：初始值、循环条件、每次循环后控制变量如何变化。

因此，**for** 循环结构更方便程序调试和代码升级。

当我们想要完善一个程序的功能，需要按编程活动步骤再来一遍。

1. 定义程序目标

问题：完善计算器程序，输入算式 $a+b$ 或 $a-b$ 或 $a*b$ 或 a/b ，程序应输出算式结果。

说明：明确程序需要哪些信息，进行什么计算，报告什么结果。

问题：如何表示运算符 $+-*/$ ？

字符类型，只处理 $+-*/$ 四个字符。

四则运算计算器的算法设计：

Begn

定义两个实数变量a, b和结果变量result;

定义字符变量oper;

从键盘接受输入的合法算式, 存入a, oper和b;

判断运算符oper的四种情况+ - * / , 然后分别

计算对应算式并将结果存入result;

显示器输出打印result的值;

End

编程活动步骤3-编写代码

四则运算计算器的算法设计:

Begn

定义两个实数变量a, b和结果变量result;

定义字符变量oper;

从键盘接受输入的合法算式, 存入a, oper和b;

判断运算符oper的四种情况+-*/，然后分别计算对应算式并将结果存入result;

显示器输出打印result的值;

End

```
/*在上一版本程序基础上修改代码  
*/
```

```
float a,b,result;
```

```
char oper;
```

```
printf("Please input  
expression as a+b or a-b or  
a*b or a/b :\n");  
scanf("%f%c%f",&a,&oper,&b);
```

完善四种判断表达式

//判断+、-、*、/四种运算符，然后分别计算。

```
if ('+' == oper){
    result = a+b;
    printf("=%f\n",result);
}else if('-' == oper){
    result = a-b;
    printf("=%f\n",result);
}else if('*' == oper){
    result = a*b;
    printf("=%f\n",result);
}else if('/' == oper){
    result = a/b;
    printf("=%f\n",result);
}else {printf("%c is Not leagal operator.\n",oper);}
return 0;
```

5. 测试和调试程序

用测试数据运行程序，查看结果是否有错。

测试数据1: 2.3, 4.9; 分别作+ - * /

测试数据2: 0, 3; 分别作+ - * /; 特别加3/0测试。

测试数据3: -3, 3; 分别作+ - * /;

测试数据4: 1234.78, 9876.54; 分别作+ - * /;

如果有错误，分析错误原因，测试确认错误原因；
然后修正代码后重新编译。

如何改善多个if-else嵌套结构

- **if**语句只有单分支**if()**和双分支**if-else**结构。
- 但是运算符变量的合法取值有4种，需要**if-else**嵌套四次，带来代码维护上的难度。
- 这种一个表达式出现多种常量取值判断的需求，适合用**switch**语句来实现。

用某个表达式单独测试每一个可能的整数值常量，然后做出相应的处理。

1. switch语句的一般形式

```
switch (表达式) { // 计算表达式的值
    case 常量表达式1: 语句组; break;
    // 等于常量式1时的处理语句组，无需加{}
    case 常量表达式2: 语句组; break;
    .....
    case 常量表达式 n: 语句组; break;
    [default: 语句组; [break; ]] // 表达式≠常量
    1..n时处理
}
```

用switch结构进行运算符判断

```
switch(oper){  
    case '+': result=a+b;  
    printf("%f%c%f=%f\n",a,oper,b,result);  
    break;  
    case '-': result=a-b;  
    printf("%f%c%f=%f\n",a,oper,b,result);  
    break;  
    case '*': result=a*b;  
    printf("%f%c%f=%f\n",a,oper,b,result);  
    break;
```

用switch结构进行运算符判断

```
    case '/': result=a/b;
    printf("%f%c%f=%f\n",a,oper,b,result);
    break;
    default:
    printf("%c is not a legal operator.\n",oper);
} //end switch
```

- 问题：输入年份和月份，计算该月有多少天？
- 算法设计：
 - 根据月份，判断大小月：**1、3、5、7、8、10、12**月是大月。
 - 根据该年是否闰年，判断**2**月的天数：闰年**29**天，非闰年**28**天。

```
#include<stdio.h>
int main(void){
    int year,month,days;
    printf("input year and month:");
    scanf("%d%d",&year,&month);
    switch(month){
        case 1:case 3: case 5:case 7: case 8:case 10: case 12:
            days = 31; break;
        case 4:case 6: case 9:case 11:
            days = 30; break;
        case 2:
            if ((year%4==0 && year%100!=0) ||
(year%400==0))
                days = 29;
            else
                days = 28;
            break;
        default:
            printf("invalid month!");}
    printf("the days are %d.\n",days);
}
```

循环结构—应用举例

求Fibonacci数列的前n项。该数列的生成方法为：
 $F_1=1$ ， $F_2=1$ ， $F_n=F_{n-1}+F_{n-2}$ （ $n \geq 3$ ），即从第3个数开始，
每个数等于前2个数之和。

Fibonacci数列示例：

1, 1, 2, 3, 5, 8, 13, 21, 34.....

求Fibonacci数列的算法设计:

Begin

```
定义int变量 f1, f2, fn, n, i;  
从键盘接受输入的整数(>=3), 存入 n;  
设置初始值: f1=1, f2=1, i=3,  
while ( i <= n)  
    fn = f1 + f2;  
    显示器打印"%d"结果值 fn;  
    /*滚动更新f1,f2 */  
    f1=f2;      f2=fn;  
    i++;
```

End

求Fibonacci数列算法1的源程序

```
#include<stdio.h>
int main(void){
    int f1, f2, fn; /*定义数列计算公式的3个变量*/
    int i,n;        /*定义循环变量i 和输入值n*/
    scanf("%d",&n);

    f1=1,f2=1;
    printf("f1=%d\nf2=%d\n", f1, f2);
    for(i=3 ; i<=n; i++ ) {
        fn = f1+f2;
        printf("f%d=%d\n",i,fn);
        f1=f2;
        f2=fn;
    }
    return 0;
}
```

5. 测试和调试程序

测试各种情况，注意条件的边界一定要测，非正常情况也要测试。

测试数据1: 10;

测试数据2: 20;

测试数据3: 30;

测试数据4: 40;

测试数据5: 50;

提示：数据溢出，应扩展数据长度。

- 基本数据类型能够被 **signed**, **unsigned**, **short**, 和 **long** 修饰. 当类型修饰符单独使用的时候, 默认的类型是 **int**.
- **unsigned int**只用于 非负值的场合。
- **short int**类型（简写为 **short**）占用的存储 空间可能比**int**类型少, 常用于较小数值的场合以节省空间。
- **long int**或 **long double**占用的存储空间可能比**int**或**double**多, 适用于较大数值的场合。
- **long long int**类型至少占64位, 适用于更大数值的场合。

求字节数运算符sizeof()

计算机程序中指定的数据类型，实际占有多少字节存储空间，决定了数据的有效范围。

单目运算符 `sizeof(数据类型名 或 变量名);`

例：

```
printf("sizeof(long) is %zu bytes.\n",sizeof(long));  
printf("sizeof(long long) is %zu bytes.\n",sizeof(long  
long));
```

求Fibonacci数列算法1的源程序

```
#include<stdio.h>
int main(void){
    long long int f1, f2, fn; /*定义数列计算公式变量*/
    int i,n;                /*定义循环变量i 和输入值n*/
    scanf("%d",&n);

    f1=1,f2=1;
    printf("f1=%d\nf2=%d\n", f1, f2);
    for(i=3 ; i<=n; i++ ) {
        fn = f1+f2;
        printf("f%d=%lld\n",i,fn);
        f1=f2;
        f2=fn;
    }
    return 0;
}
```

5. 测试和调试程序

测试各种情况，注意条件的边界一定要测，非正常情况也要测试。

测试数据1: 50;

测试数据2: 60;

测试数据3: 70;

测试数据4: 80;

测试数据5: 90;

测试数据6: 100;

Fib数列算法改进：每循环计算两项

求Fibonacci数列的改进算法：

Begin

定义long lon int变量 f1, f2;

定义int变量 n, i;

从键盘接受输入的整数(≥ 3), 存入 n;

设置初始值: f1=1, f2=1, i=1,

while (i $\leq$ n)

显示器打印已知两项值 f1, f2;

/*滚动更新f1,f2 */

f1 = f1 + f2;

f2 = f1 + f2;

i += 2;

End



求Fibonacci数列改进算法

```
#include<stdio.h>
int main(void){
    long long int f1, f2; /*定义数列计算公式的变量*/
    int i,n;             /*定义循环变量i 和输入值n*/
    scanf("%d",&n);

    for(f1=1,f2=1,i=1; i<=n; i+=2) {
        printf("f%d=%lld\n",i,f1);
        if (i+1 <= n)
            printf("f%d=%lld\n",i+1,f2);
        f1 = f1+f2;
        f2 = f1+f2;
    }
    return 0;
}
```

5. 测试和调试程序

测试各种情况，注意条件的边界一定要测，非正常情况也要测试。

测试数据1: 11;

测试数据2: 50;

测试数据3: 70;

测试数据4: 90;

测试数据5: 93;

问题：打印直角三角形

输入一个正整数作为给定直角三角形的最大行数，
要求程序用*号打印出直角三角形。

样例输入： 7

样例输出： *

**

打印直角三角的算法设计

打印直角三角形的算法设计：

Begn

定义整数变量lines;

从键盘接受输入的正整数，存入lines;

定义整数变量i和j表示当前打印的行号和列号;

while (i不超过lines)

打印第i行的i个星号;

打印换行符;

行号+1;

End

完善：打印变量指定的i个星号。

列号j可以用来计数已打星号个数;

while(j不超过i)

打印星号;

编写打印直角三角程序

```
#include <stdio.h>
int main(void){
    int lines, i,j;

    printf("Enter a positive integer:");
    scanf("%d",&lines);

    for (i=1; i <= lines; i++){
        //打印第i行i个*号
        for (j=1; j <= i; j++)
            putchar('*');
        putchar('\n'); //printf("\n");
    }
    return 0;
}
```

- 自加运算符`++`和自減运算符`--`是C语言特有的运算符，为了简化变量自加減1的表达式书写。
- `x++;` `//` 表达式`x=x+1;`的简写
- `x--;` `//` 表达式`x=x-1;`的简写
- 注意`++`和`--`有前綴和后綴的区别：
 - `++ i`, `-- i` (在使用 `i` 之前, 先使 `i` 的值加(減) 1)
 - `i ++`, `i --` (在使用 `i` 之后, 使 `i` 的值加(減) 1)

4. 编译

目的：检查程序有效性（符合编程语言语法）。

5. 测试和调试程序

测试各种情况，注意条件的边界一定要测，非正常情况也要测试。

测试数据1： 1；

测试数据2： 8；

测试数据3： 0；

测试数据4： -1；

测试数据5： 12；

问题：凯撒密码

输入一行字符，将其中所有的字母都修改为字符表中右移3位后的字符，空格和其它符号不变。然后将修改后的密文字符输出。

样例输入： **hi, tom!**

样例输出： **kl, wrp!**

样例输入： **xyz**

样例输出： **abc**

如何使完成的程序重复执行呢？

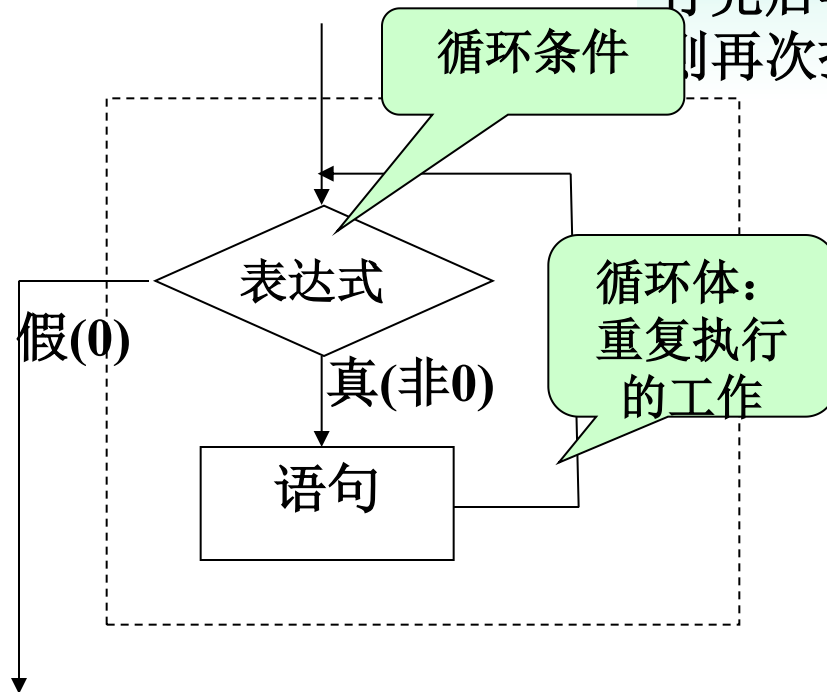
四、循环结构语句

- ◆ 有时程序中需要多次运行同一段代码（重复做相同的事情）。这种控制结构称为循环结构。
- ◆ 在循环结构中，需要刻画出重复执行的是哪些动作（循环体），以及什么条件下需要重复（循环条件）。
- ◆ C语言提供了三类用于实现循环结构的语句：
 - 1) while
 - 2) do-while
 - 3) for

此处的表达式可以为任何一种表达式,语句为任意类语句;

如果表达式值为真(非0), 则执行语句; 执行完后再测试表达式, 如果仍为真(非0), 则再次执行语句; 直到表达式为假(为0)。

while



a) 流程图

```
while (表达式)
    语句
```

b) C语句

图3. 6 while语句

凯撒加密的算法设计：

Begn

定义字符变量letter;

从键盘接受输入的字符，存入letter;

while(letter不是行结束符)

if(letter是大写字母或小写字母)

显示器输出打印letter+3的字符;

else 显示器输出打印原letter;

从键盘接受输入的字符，存入letter;

End

完善：letter+3, 对于字母表末尾的字母会出错吗？'X'+3=?

解决思路：模运算适合表达固定边界的整数运算。

问题: `letter+3`, 对于字母表末尾的字母会越界!

解决思路: 模运算适合表达固定边界的整数运算。

分析: ‘X’ +3 如何映射到 ‘A’ ?

字母表内部序号0-25, 在模26上加运算可以完成:

$$(23 + 3) \% 26 = 26 \% 26 = 0.$$

算法: 只要将字母先转成字母表序号, 模加后再从字母表序号转回字母就达到目标。

表达式实现: 大写字母转序号 `(letter - 'A')`;

$$((\text{letter} - \text{'A'}) + 3) \% 26 + \text{'A'};$$

凯撒加密的算法完善:

Begn

定义字符变量letter;

从键盘接受输入的字符, 存入letter;

while(letter不是行结束符)

if(letter是大写字母)

打印 $((\text{letter} - 'A') + 3) \% 26 + 'A'$ 的字符;

else if(letter是小写字母)

打印 $((\text{letter} - 'a') + 3) \% 26 + 'a'$ 的字符;

else

显示器输出打印原letter;

从键盘接受输入的字符, 存入letter;

End

```
#include <stdio.h>
#define DIFF 3
int main(void){
    printf("This is a Caesar Cipher.\n");
    char letter;
    for(letter=getchar(); letter!='\n'; letter=getchar()){
        //for (; (letter=getchar())!='\n'; ){
            if (letter >='A' && letter<='Z')
                putchar((letter-'A'+ DIFF)%26+'A');
            else if(letter >='a' && letter<='z')
                putchar((letter-'a'+ DIFF)%26+'a');
            else
                putchar(letter);
        }
    }
    putchar('\n');    return 0;
}
```

4. 编译

目的：检查程序有效性（符合编程语言语法）。

5. 测试和调试程序

测试各种情况，注意条件的边界一定要测，非正常情况也要测试。

测试数据1: hi,tom. ; 测试数据2: xyz?;

测试数据3: Eat a apple!;

测试数据3: start at 9 clock. ;

赋值和逻辑判断的简写

```
letter=getchar();  
while (letter!='\n'){  
    ...  
    letter=getchar();  
}
```

C语言中，赋值表达式用小括号括起来之后，可以代表所赋变量，继续和逻辑表达式嵌套组合。

因此，上面三行代码可以简写为一行：

```
while ((letter=getchar())!='\n'){...}
```

